

Binäre Suche – Integer-Array (Winter 2021/22)

Pseudocode (IHK-Stil):

```
Funktion binarySearch(werte: Array<Integer>, x: Integer) → Integer
    links = 0
    rechts = länge(werte) - 1
    while links ≤ rechts
        mitte = (links + rechts) / 2
        if werte[mitte] == x dann
            return mitte
        else if werte[mitte] < x dann
            links = mitte + 1
        else
            rechts = mitte - 1
        end if
    end while
    return -1
end Funktion
```

Java-Beispiel:

```
public static int binarySearch(int[] werte, int x) {
    int links = 0, rechts = werte.length - 1;
    while (links <= rechts) {
        int mitte = (links + rechts) / 2;
        if (werte[mitte] == x) return mitte;
        else if (werte[mitte] < x) links = mitte + 1;
        else rechts = mitte - 1;
    }
    return -1;
}
```

Erklärung:

Voraussetzung: Array ist ****aufsteigend sortiert****.

Ablauf: In jedem Schritt halbiert du den Suchbereich. $Mitte = (links + rechts) / 2$.

Beispiel: Suche 8 in [2,4,6,8,10]. Mitte=2 (Wert=6) < 8 → rechts weiter. Neue Mitte=3 (Wert=8) = Treffer.

Komplexität: $O(\log n)$.

Typische Fehler: Schleifenbedingung falsch, Mitte nicht ganzzahlig, Sortierung vergessen.

Prüfungstipp: Immer erwähnen „Array muss sortiert sein“.

Binäre Suche – erstes Vorkommen bei Duplikaten

Pseudocode (IHK-Stil):

```
Funktion binarySearchErstes(werte: Array<Integer>, x: Integer) → Integer
    links = 0
    rechts = länge(werte) - 1
    ergebnis = -1
    while links ≤ rechts
        mitte = (links + rechts) / 2
        if werte[mitte] == x dann
            ergebnis = mitte
            rechts = mitte - 1
        else if werte[mitte] < x dann
            links = mitte + 1
        else
            rechts = mitte - 1
        end if
    end while
    return ergebnis
end Funktion
```

Java-Beispiel:

```
public static int binarySearchErstes(int[] a, int x) {
    int l = 0, r = a.length - 1, res = -1;
    while (l <= r) {
        int m = (l + r) / 2;
        if (a[m] == x) { res = m; r = m - 1; }
        else if (a[m] < x) l = m + 1;
        else r = m - 1;
    }
    return res;
}
```

Erklärung:

Ziel: Das ****erste Vorkommen**** finden, wenn ein Wert mehrfach vorkommt.

Ablauf: Bei Gleichheit speicherst du den Index, gehst aber links weiter (rechts = mitte-1).

Beispiel: [2,4,4,4,6], Suche 4:

- mitte=2 (Wert=4) → Treffer bei Index 2 → speichere=2 → suche links weiter.
- neue Mitte=0 (Wert=2) < 4 → gehe rechts → Index 1 (Wert=4) → Treffer.
- Ergebnis = 1.

Komplexität: $O(\log n)$.

Typische Fehler: Sofort return bei Treffer (führt nicht zum ersten Element).

Prüfungstipp: Immer erklären, warum man weiter links suchen muss.

Lineare Suche – String-Liste (case-insensitiv)

Pseudocode (IHK-Stil):

```
Funktion findeName(L: Liste<String>, s: String) → Integer
    such = lower(s)
    for i = 0 to L.size() - 1
        if lower(L.get(i)) == such dann
            return i
        end if
    end for
    return -1
end Funktion
```

Java-Beispiel:

```
public static int findeName(java.util.List<String> liste, String s) {
    String such = s.toLowerCase();
    for (int i = 0; i < liste.size(); i++) {
        if (liste.get(i).toLowerCase().equals(such)) return i;
    }
    return -1;
}
```

Erklärung:

Vergleicht Strings ****ohne Beachtung der Groß-/Kleinschreibung****.

Ablauf: Wandle gesuchten Namen und alle Listeneinträge mit toLowerCase() um.

Beispiel: Liste=["Anna","BEN","Chris"], Suche "ben":

- i=0 → "anna" != "ben"

- i=1 → "ben" == "ben" → Rückgabe Index 1.

Komplexität: $O(n)$.

Typische Fehler: Letztes Element nicht prüfen; equals statt toLowerCase() vergessen.

Prüfungstipp: Bei Strings in Prüfungen immer überlegen, ob Case-Sensitivity relevant ist.